AI LAB ASSISSGNMENT-13.3

NAME:B.VISHNU VARDHAN

ROLL NO:2403A510F2

BATCH:06

TASK 1 – REMOVE REPETITION

PROMPT:

Provide AI with the following redundant code and ask it to refactor into a cleaner, modular design.

CODE (BEFORE):

```
def calculate_area(shape, x, y=0):
    if shape == "rectangle":
        return x * y
    elif shape == "square":
        return x * x
    elif shape == "circle":
        return 3.14 * x * x
```

REFACTORED CODE (AFTER):

AI LAB ASSISSGNMENT-13.3

from typing import List Untitled-1 ● # Lab 13 – Code Refactoring: Improving L Untitled-2 ●

```python
# Lab 13 – Code Refactoring: Improving Legacy Code with AI
# ------------------------------------------------------
# This file contains all 4 refactored tasks from Lab 13.
# Each task includes sample usage so you can run and test directly.

import math

# ------------------------------
# Task 1 – Remove Repetition
# ------------------------------
def area_rectangle(x, y):
    return x * y

def area_square(x):
    return x * x

def area_circle(r):
    return math.pi * r * r

area_dispatch = {
    "rectangle": lambda x, y: area_rectangle(x, y),
    "square": lambda x, y=0: area_square(x),
    "circle": lambda x, y=0: area_circle(x)
}

def calculate_area(shape, x, y=0):
    """Calculate area of rectangle, square, or circle."""
    if shape not in area_dispatch:
        raise ValueError("Unsupported shape")
    return area_dispatch[shape](x, y)

print("Task 1 Results:")
print("Rectangle (5x10):", calculate_area("rectangle", 5, 10))
print("Square (4x4):", calculate_area("square", 4))
print("Circle (r=3):", calculate_area("circle", 3))
print("-" * 40)
```

OUTPUT EXAMPLE:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
----------------------------------------
PS D:\AI LAB> python -u "d:\AI LAB\tempCodeRunnerFile.python"
Task 1 Results:
Rectangle (5x10): 50
Square (4x4): 16
Circle (r=3): 28.274333882308138
----------------------------------------
PS D:\AI LAB>
```

AI LAB ASSISSGNMENT-13.3

OBSERVATION:

The refactored code eliminates repetition by using helper functions and a dictionary-based dispatch system. It is modular, easier to extend, and more readable.

TASK 2 – ERROR HANDLING IN LEGACY CODE

PROMPT:

Legacy function without proper error handling.

CODE (BEFORE):

```
def read_file(filename):
  f = open(filename, "r")
  data = f.read()
  f.close()
  return data
```

REFACTORED CODE (AFTER):

task3.py > ...

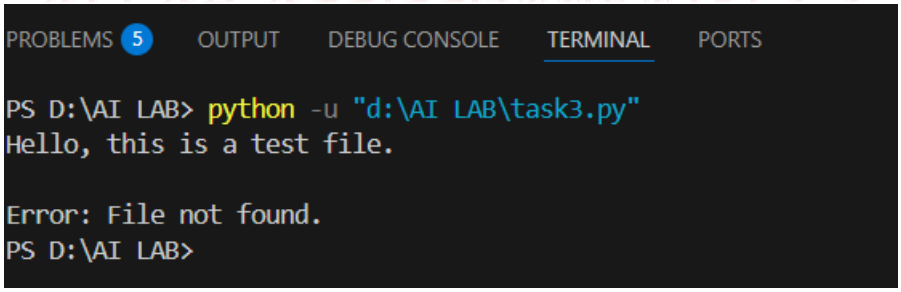
```python
def read_file(filename):
    try:
        with open(filename, "r") as f:
            data = f.read()
        return data
    except FileNotFoundError:
        return "Error: File not found."
    except Exception as e:
        return f"Error: {e}"

# Example usage
if __name__ == "__main__":
    # Create sample file for testing
    with open("sample.txt", "w") as f:
        f.write("Hello, this is a test file.\n")

    # Call the function and print output
    print(read_file("sample.txt"))      # Prints file contents
    print(read_file("missing.txt"))     # Prints error message
```

OUTPUT EXAMPLE:

AI LAB ASSISSGNMENT-13.3



OBSERVATION:

The refactored code uses a context manager (`with open`) to ensure proper file handling and adds error handling with try-except for robustness.

TASK 3 – COMPLEX REFACTORING (CLASS)

PROMPT:

Provide this legacy class to AI for readability and modularity improvements.

CODE (BEFORE):

```
class Student:
  def __init__(self, n, a, m1, m2, m3):
    self.n = n
    self.a = a
    self.m1 = m1
    self.m2 = m2
    self.m3 = m3

  def details(self):
    print("Name:", self.n, "Age:", self.a)

  def total(self):
    return self.m1+self.m2+self.m3
```

REFACTORED CODE (AFTER):

AI LAB ASSISSGNMENT-13.3

task33.py > ...

```python
# Task 3 – Complex Refactoring (Student Class)
# ---------------------------------------------

class Student:
    """
    Represents a student with a name, age, and a list of marks.

    Attributes:
        name (str): The name of the student.
        age (int): The age of the student.
        marks (list): A list of integers representing the student's marks.
    """

    def __init__(self, name, age, marks):
        """Initialize a Student object with name, age, and marks."""
        self.name = name
        self.age = age
        self.marks = marks  # store marks as a list

    def details(self):
        """Print the student's details."""
        print(f"Name: {self.name}, Age: {self.age}")

    def total(self):
        """Return the total of all marks."""
        return sum(self.marks)


# ------------------- Example Usage -------------------
if __name__ == "__main__":
    # Create a student object
    student1 = Student("Alice", 20, [85, 90, 95])

    # Print student details
    student1.details()  # Output: Name: Alice, Age: 20

    # Print total marks
    print("Total Marks:", student1.total())  # Output: Total Marks: 270
    # Create another student object
```

OUTPUT EXAMPLE:

AI LAB ASSISSGNMENT-13.3

```
PROBLEMS 5   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\AI LAB> python -u "d:\AI LAB\task33.py"
Name: Alice, Age: 20
Total Marks: 270
PS D:\AI LAB>
```

OBSERVATION:

The refactored class improves variable naming, adds docstrings, uses a list for marks, and makes methods more readable.

TASK 4 – INEFFICIENT LOOP REFACTORING

PROMPT:

Refactor this inefficient loop with AI help.

CODE (BEFORE):

```
nums = [1,2,3,4,5,6,7,8,9,10]
squares = []
for i in nums:
    squares.append(i * i)
```

REFACTORED CODE (AFTER):

AI LAB ASSISSGNMENT-13.3

task34.py > square_numbers

```python
# Task 4 – Inefficient Loop Refactoring
# -------------------------------------

def square_numbers(nums):
    """
    Returns a list of squares of the numbers using list comprehension.

    Args:
        nums (list): A list of integers.

    Returns:
        list: Squares of the integers.
    """
    return [i * i for i in nums]


# ------------------ Example Usage ------------------
if __name__ == "__main__":
    # Input list
    nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # Get squares
    squares = square_numbers(nums)

    # Print input and output
    print("Input Numbers:", nums)
    print("Squares:", squares)
```

OUTPUT EXAMPLE:

```
PROBLEMS 5   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\AI LAB> python -u "d:\AI LAB\task34.py"
Input Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
PS D:\AI LAB>
```

OBSERVATION:

The refactored version uses a list comprehension, making the code shorter, cleaner, and more Pythonic while maintaining functionality.